

Machine Learning



Dr. Ahmed Hesham Mostafa
Faculty of Computers and AI
Helwan University
Lecture 3

Course Staff: Instructors (Short Bio)

- **Dr. Ahmed Hesham Mostafa**
- Bachelor in Computer Science *from* Helwan University
- Masters in Computer Science (Malware Analysis & Artificial Intelligence) *from* Helwan University
- PhD. in Computer Science (Computer Vision & Artificial Intelligence) *from* Helwan University
- E-mail : ahmedheshamostafa@gmail.com

Optimization

- In machine learning, optimization refers to the process of adjusting the parameters of a model to minimize or maximize a certain objective function.
- The objective function represents a measure of how well the model is performing on a given task, such as classification accuracy, prediction error, or any other relevant metric.
- Optimization is crucial because the performance of machine learning models heavily depends on the values of their parameters. By finding the optimal values for these parameters, we aim to improve the model's ability to generalize to new, unseen data.

Process of Optimization

- 1. Define an Objective Function:** This function quantifies how well the model is performing on the task at hand. For example, in a classification problem, the objective function could be the cross-entropy loss, while in a regression problem, it could be the mean squared error.
- 2. Choose an Optimization Algorithm:** There are various optimization algorithms available, each with its own strengths and weaknesses. Some common optimization algorithms used in machine learning include gradient descent, stochastic gradient descent (SGD), Adam, RMSprop, and others.
- 3. Update Model Parameters:** The optimization algorithm iteratively adjusts the model's parameters to minimize or maximize the objective function. This involves computing the gradient of the objective function with respect to the parameters and using it to update the parameters in the direction that reduces the objective function.
- 4. Convergence:** The optimization process continues until a stopping criterion is met, such as reaching a maximum number of iterations, or when the change in the objective function falls below a certain threshold. At this point, the model parameters are considered to have converged to their optimal values.

Process of optimization

- Here are the basic steps involved in optimization in machine learning along with their corresponding formula symbols:
- 1. Define the Objective Function (Loss Function):
 - - Objective function ($J(\theta)$), where (θ) represents the parameters of the model.
- 2. Choose an Optimization Algorithm:
 - - There are various optimization algorithms available such as Gradient Descent, Stochastic Gradient Descent, Adam, RMSprop, etc.

Process of optimization

- 3. Compute the Gradient:
 - - Gradient of the objective function with respect to the parameters: ($\nabla J(\theta)$).
- 4. Update the Parameters:
 - - Parameter update rule: ($\theta_{\{t+1\}} = \theta_{\{t\}} - \alpha \cdot \nabla J(\theta_{\{t\}})$), where (α) is the learning rate.
- 5. Repeat Steps 3-4 until Convergence:
 - - Iterate through steps 3 and 4 until the algorithm converges, i.e., until the change in the objective function or the parameters falls below a predefined threshold or a certain number of iterations is reached.

Evolution of Optimization Algorithms

Gradient Descent:

1. Gradient descent is a first-order optimization algorithm used to minimize an objective function by iteratively moving in the direction of the steepest descent of the function.
2. It computes the gradient of the objective function with respect to all parameters and updates the parameters in the opposite direction of the gradient.

Evolution of Optimization Algorithms

Stochastic Gradient Descent (SGD):

1. Stochastic Gradient Descent is a variant of gradient descent that computes the gradient using a single or a small subset of randomly selected samples from the training dataset.
2. It introduces randomness into the optimization process, which helps in reducing computational cost and escaping local minima.

Momentum:

- Introduction: Momentum adds a fraction of the previous update vector to the current update, allowing the optimization process to continue moving in the same direction for consecutive iterations.

AdaGrad:

1. AdaGrad (Adaptive Gradient Algorithm) is an adaptive learning rate optimization algorithm that adapts the learning rates of model parameters based on the historical gradients.
2. It divides the learning rate for each parameter by the square root of the sum of squared gradients for that parameter, effectively giving larger updates to parameters with infrequent updates and smaller updates to frequently updated parameters.

Evolution of Optimization Algorithms

RMSprop:

1. RMSprop (Root Mean Square Propagation) is a variant of AdaGrad that addresses its weakness of diminishing learning rates.
2. It replaces the sum of squared gradients in AdaGrad with an exponentially decaying average of squared gradients, which prevents the learning rates from becoming too small too quickly.

Adam (Adaptive Moment Estimation):

1. Adam is an adaptive learning rate optimization algorithm that combines ideas from momentum optimization and RMSprop.
2. It maintains both a decaying average of past gradients and past squared gradients, and adjusts the learning rates for each parameter accordingly.
3. Adam is known for its efficiency, robustness, and ability to handle noisy gradients and non-stationary objectives.

Stochastic Gradient Descent (SGD)

- Stochastic Gradient Descent (SGD) is a variant of the standard gradient descent optimization algorithm commonly used in machine learning and optimization. It is particularly well-suited for large datasets and online learning scenarios. Here's an overview of stochastic gradient descent:
- Stochastic Gradient Descent (SGD) is an iterative optimization algorithm used to minimize an objective function (e.g., a loss function) by adjusting the parameters of a model.
- Unlike traditional gradient descent, which computes the gradient of the loss function using the entire dataset, SGD computes the gradient using only one randomly chosen training example (or a small subset of examples) in each iteration.

Stochastic Gradient Descent (SGD)

- **Algorithm:**
- At each iteration:
 - Randomly shuffle the training dataset.
 - Select one training example (or a mini-batch of examples) from the dataset.
 - Compute the gradient of the loss function with respect to the parameters using the selected training example(s).
 - Update the parameters using the computed gradient and a predefined learning rate.

The gradient of "one-example-loss" might point in a different direction, and might help us steer clear of these (i.e., escape local minima)

Stochastic Gradient Descent (SGD)

Advantages:

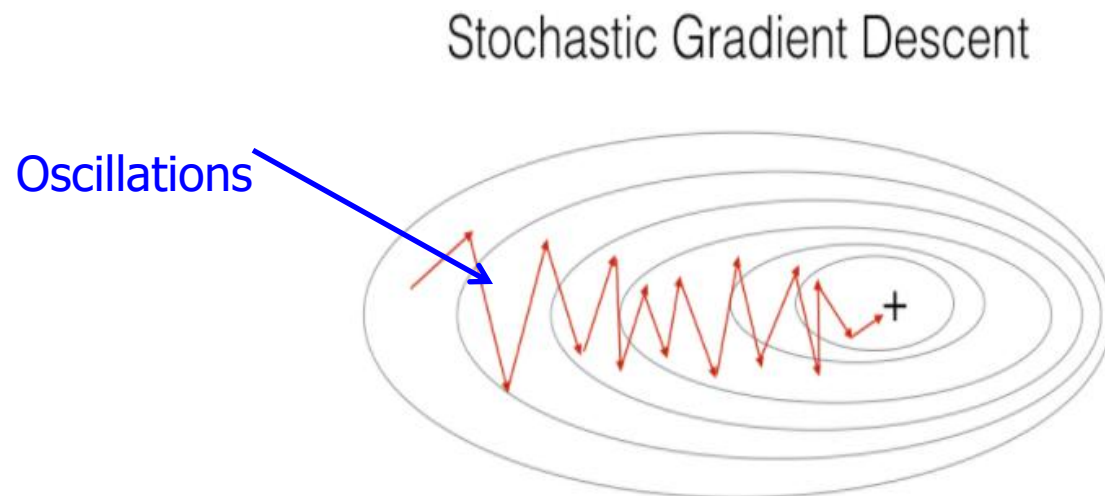
- 1. Efficiency:** SGD is computationally more efficient compared to batch gradient descent, especially for large datasets, as it processes only one (or a small subset of) training example(s) in each iteration.
- 2. Online Learning:** SGD is well-suited for online learning scenarios where new data becomes available continuously, as it can update the model's parameters incrementally with each new training example.

Challenges:

- 1. High Variance:** SGD can exhibit high variance in parameter updates due to the randomness introduced by using only one training example at a time. This can lead to noisy convergence and slower convergence in some cases.
- 2. Learning Rate Selection:** The choice of learning rate in SGD is critical. A learning rate that is too large can lead to instability and overshooting, while a learning rate that is too small can result in slow convergence.

Stochastic Gradient Descent

- Due to the way how data is used to calculate the derivative of the loss function → **oscillations** occur while taking steps towards the global minima which help avoid getting stuck in local minima
- The gradient of "one-example-loss" might **point in a different direction**, and might help us steer clear of these (i.e., escape local minima)
- The frequent updates create **plenty of oscillations** which can be helpful for **getting out of** local minimums.



Problems with oscillations

1.Instability:

1. Oscillations introduce instability into the optimization process, making it challenging to predict the behavior of the algorithm.
2. Unstable optimization trajectories can make it difficult to assess the progress of the optimization and to determine when convergence has been reached.

2.Difficulty in Finding Optimal Parameters:

1. Oscillations can hinder the algorithm's ability to accurately estimate the optimal parameters of the model.
2. Rapid changes in parameter values can lead to overshooting "jump over" or undershooting "too small" the optimal solution, making it challenging to converge to the true minimum of the objective function.

Stochastic Gradient Descent (SGD)

- The update rule for parameters is as follows:
- $[\theta := \theta - \alpha \frac{1}{m} \sum_{i=1}^m \nabla J(\theta; x^{(i)}, y^{(i)})]$
- Where:
- (m) is the total number of training examples.
- $(\nabla J(\theta; x^{(i)}, y^{(i)}))$ is the gradient of the loss function with respect to the parameters for *the* (i) *th* training example.

Stochastic Gradient Descent (SGD)

Example

- example with a dataset containing four features (x_1, x_2, x_3, x_4) and one target variable (y):
- We want to fit a linear regression model with the hypothesis function:
- $h_{\theta}(x) = \theta_0 + \theta_1 \times x_1 + \theta_2 \times x_2 + \theta_3 \times x_3 + \theta_4 \times x_4$
- Let's assume we initialize the parameters randomly: $\theta_0 = 0.5, \theta_1 = 0.5, \theta_2 = 0.5, \theta_3 = 0.5$, and $\theta_4 = 0.5$, and set the learning rate $\alpha = 0.01$.
- We'll use Mean Squared Error (MSE) as the loss function:
- $J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$
- Now, let's go through one iteration of SGD:

Stochastic Gradient Descent (SGD)

Example

- 1. Select Random Data Point: Let's say we randomly select the first data point (1, 2, 3, 4, 5). y actual
- 2. Forward Pass: Calculate the predicted output using the current parameters:
 - $h_{\theta}(1,2,3,4) = 0.5 + 0.5 \times 1 + 0.5 \times 2 + 0.5 \times 3 + 0.5 \times 4 \approx 6$
- 3. Compute Loss: Calculate the loss function (MSE) for the selected data point:
- $J(\theta) = \frac{1}{2} \times (6 - 5)^2 = 0.5$

Stochastic Gradient Descent (SGD)

Example

- To compute the gradients for each parameter θ_0 , θ_1 , and θ_2 , we'll use the formula:
- $\frac{\partial J}{\partial \theta_i} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_i^{(i)}$
- For θ_0 :
- $\frac{\partial J}{\partial \theta_0} = \frac{1}{3} \sum_{i=1}^3 (h_{\theta}(x^{(i)}) - y^{(i)})$
- For θ_1 :
- $\frac{\partial J}{\partial \theta_1} = \frac{1}{3} \sum_{i=1}^3 (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_1^{(i)}$
- For θ_2 :
- $\frac{\partial J}{\partial \theta_2} = \frac{1}{3} \sum_{i=1}^3 (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_2^{(i)}$

Mean Square Error

- $J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x_i) - y_i)^2$
- The Mean Square Error(MSE) or L2 loss is a loss function that quantifies the magnitude of the error between a machine learning algorithm prediction and an actual output by taking the average of the squared difference between the predictions and the target values. Squaring the difference between the predictions and actual target values results in a higher penalty assigned to more significant deviations from the target value. A mean of the errors normalizes the total errors against the number of samples in a dataset or observation.

A partial derivative

- Chain Rule: If $f(x) = g(h(x))$, then $f'(x) = g'(h(x)) \cdot h'(x)$. This rule allows us to differentiate composite functions.

- 1. Single Variable Chain Rule: This is the standard chain rule for functions of one variable. If $y = f(u)$ and $u = g(x)$, then the chain rule states:

- $$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

- 2. Generalized Chain Rule (Single Variable): For functions involving multiple compositions, the chain rule extends as follows. If $y = f(u)$, $u = g(v)$, and $v = h(x)$, then:

- $$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dv} \cdot \frac{dv}{dx}$$

A partial derivative

- A partial derivative is a concept from calculus used to calculate the rate at which a function changes with respect to one of its independent variables, while holding all other variables constant. It's a way to understand how a function changes in one direction without considering the effects of changes in other directions.
- Let's break down the definition further:
- Function of Several Variables: Partial derivatives are used when dealing with functions of several variables. For example, $f(x, y)$ could represent a function that depends on both x and y .

A partial derivative

- So, we need partial derivative in loss to show how the change of one input or parameters can effect in changing in the loss
- How change in the weight w with considering b is constant can change and effect in the loss and vice versa

Derivative of Mean Square Error

- $J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x_i) - y_i)^2$
- Assume the hypotheses function is
- $h_{\theta}(x) = b_i + w_i \times x_i$
- Let's calculate the Derivative of MSE respected to W
- As MSE is composite function of $h_{\theta}(x)$ so we will use chain rule
- $\frac{\partial J}{\partial w_i} = \frac{\partial J}{\partial h_{\theta}} \times \frac{\partial h_{\theta}}{\partial w_i}$

Derivative of Mean Square Error

- $\frac{\partial J}{\partial h_{\theta}} = \frac{\partial}{\partial h_{\theta}} \left(\frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x_i) - y_i)^2 \right)$
- $\frac{\partial J}{\partial h_{\theta}} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x_i) - y_i)$
- $\frac{\partial h_{\theta}}{\partial w_i} = \frac{\partial}{\partial w_i} (b_i + w_i x_i)$
- $\frac{\partial h_{\theta}}{\partial w_i} = x_i$
- So $\frac{\partial J}{\partial w_i} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x_i) - y_i) \quad x_i$

Derivative of Mean Square Error

- Derivative respect to b
- $\frac{\partial J}{\partial b_i} = \frac{\partial J}{\partial h_\theta} \times \frac{\partial h_\theta}{\partial b_i}$
- $\frac{\partial h_\theta}{\partial b_i} = \frac{\partial}{\partial w_i} (b_i + w_i x_i)$
- $\frac{\partial h_\theta}{\partial b_i} = 1$
- So $\frac{\partial J}{\partial b_i} = \frac{1}{m} \sum_{i=1}^m (h_\theta(x_i) - y_i)$

Stochastic Gradient Descent (SGD)

Example

- To compute the gradients for each parameter θ_0 , θ_1 , and θ_2 , we'll use the formula:
- $\frac{\partial J}{\partial \theta_i} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_i^{(i)}$
- For θ_0 :
- $\frac{\partial J}{\partial \theta_0} = \frac{1}{3} \sum_{i=1}^3 (h_{\theta}(x^{(i)}) - y^{(i)})$
- For θ_1 :
- $\frac{\partial J}{\partial \theta_1} = \frac{1}{3} \sum_{i=1}^3 (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_1^{(i)}$
- For θ_2 :
- $\frac{\partial J}{\partial \theta_2} = \frac{1}{3} \sum_{i=1}^3 (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_2^{(i)}$

Stochastic Gradient Descent (SGD)

Example

- Compute the gradients of the loss function with respect to the parameters:
- $\frac{\partial J(\theta)}{\partial \theta_0} = (h_{\theta}(1,2,3,4) - y^{(1)}) \times 1 = (6 - 5) \times 1 = 1$
- $\frac{\partial J(\theta)}{\partial \theta_1} = (h_{\theta}(1,2,3,4) - y^{(1)}) \times x_1^{(1)} = (6 - 5) \times 1 = 1$
- $\frac{\partial J(\theta)}{\partial \theta_2} = (h_{\theta}(1,2,3,4) - y^{(1)}) \times x_2^{(1)} = (6 - 5) \times 2 = 2$
- $\frac{\partial J(\theta)}{\partial \theta_3} = (h_{\theta}(1,2,3,4) - y^{(1)}) \times x_3^{(1)} = (6 - 5) \times 3 = 3$
- $\frac{\partial J(\theta)}{\partial \theta_4} = (h_{\theta}(1,2,3,4) - y^{(1)}) \times x_4^{(1)} = (6 - 5) \times 4 = 4$

Stochastic Gradient Descent (SGD)

Example

- Update Parameters: Update the parameters using the gradients and learning rate:
- $\theta_0 = \theta_0 - \alpha \times \frac{\partial J(\theta)}{\partial \theta_0} = 0.5 - 0.01 \times 1 = 0.49$
- $\theta_1 = \theta_1 - \alpha \times \frac{\partial J(\theta)}{\partial \theta_1} = 0.5 - 0.01 \times 1 = 0.49$
- $\theta_2 = \theta_2 - \alpha \times \frac{\partial J(\theta)}{\partial \theta_2} = 0.5 - 0.01 \times 2 = 0.48$
- $\theta_3 = \theta_3 - \alpha \times \frac{\partial J(\theta)}{\partial \theta_3} = 0.5 - 0.01 \times 3 = 0.47$
- $\theta_4 = \theta_4 - \alpha \times \frac{\partial J(\theta)}{\partial \theta_4} = 0.5 - 0.01 \times 4 = 0.46$
- This completes one iteration of SGD. We would repeat these steps for each data point in the dataset for multiple epochs until convergence.

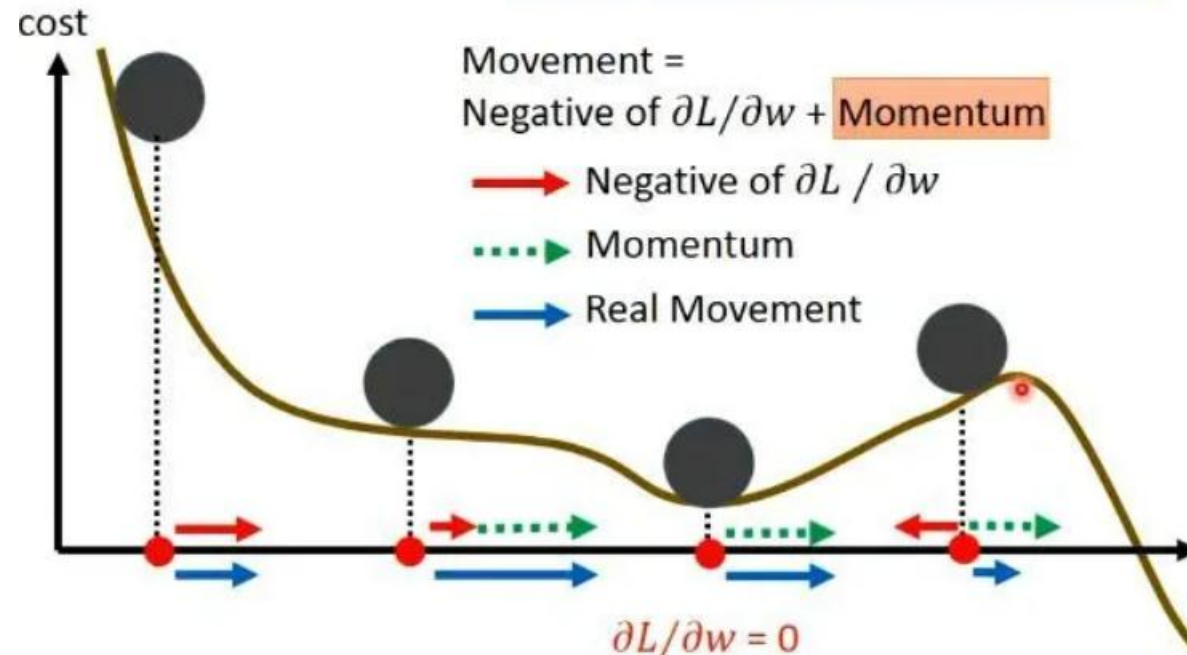
Momentum

- Momentum, in the context of optimization algorithms used in machine learning, is a technique that aims to accelerate convergence during the training process.
- The idea behind momentum is inspired by concepts from physics, where momentum represents the "velocity" of a moving object.
- In optimization, momentum introduces a momentum term that accumulates gradients over time and influences the direction and speed of parameter updates.

Momentum

Momentum

Still not guarantee reaching global minima, but give some hope



Momentum

- The parameter update rule with momentum can be expressed as follows:
- $v_t = \beta \cdot v_{t-1} + (1 - \beta) \cdot \nabla J(\theta_t)$
- $\theta_{t+1} = \theta_t - \alpha \cdot v_t$
- Where: - θ_t is the parameter at iteration t . -
- α is the learning rate. -
- β is the momentum coefficient (usually a value close to 1).
- v_t is the velocity at iteration t .
- - $\nabla J(\theta_t)$ is the gradient of the loss function with respect to the parameters at iteration t .

Momentum

- In traditional optimization algorithms like gradient descent, each parameter update is solely based on the current gradient. This can lead to slow convergence, especially in regions with high curvature or small gradients.
- Momentum introduces a "velocity" term that accelerates parameter updates by taking into account past gradients. This momentum term provides a form of "memory" that helps the algorithm maintain its direction and speed, leading to faster convergence.
- In regions with high curvature or noise, gradient descent algorithms may oscillate or zig-zag around the minimum, slowing down convergence.
- Momentum helps dampen these oscillations by providing a smoothing effect. It averages out the gradients over time, reducing the impact of sudden changes in gradient direction and making parameter updates more consistent.

Momentum

- In flat regions of the optimization landscape, where gradients are small or close to zero, traditional optimization algorithms tend to converge slowly. - Momentum enables the algorithm to maintain its momentum even in such regions. By accumulating past gradients, the momentum term allows the algorithm to "carry forward" momentum from previous updates, enabling faster traversal through flat regions.
- The momentum term is typically computed as a weighted average of past gradients, where the weight assigned to each gradient is determined by a momentum coefficient (usually denoted as (β)).
- A higher momentum coefficient assigns more weight to past gradients, resulting in smoother and more continuous updates.
- a lower momentum coefficient gives less weight to past gradients, resulting in more responsive but potentially noisy updates.

Momentum

- Let's illustrate the effect of momentum using a simple numerical example of gradient descent with and without momentum. We'll consider optimizing a quadratic function:
- $f(x) = x^2$
- The goal of gradient descent is to find the minimum of this function. We'll compare the performance of standard gradient descent (without momentum) and gradient descent with momentum in terms of convergence speed and stability.
- For simplicity, let's start with an initial guess $x_0 = 4$ and a learning rate $\alpha = 0.1$. We'll iterate for a few steps and observe the behavior of both approaches.

Momentum Example

- **1. Gradient Descent (Without Momentum):**

- $x_{t+1} = x_t - \alpha \cdot \frac{\partial f}{\partial x}$

- $x_{t+1} = x_t - \alpha \cdot 2x_t$

- $x_{t+1} = x_t - 0.1 \cdot 2x_t$

- **2. Gradient Descent with Momentum:**

- $v_{t+1} = \beta \cdot v_t + (1 - \beta) \cdot \frac{\partial f}{\partial x}$

- $x_{t+1} = x_t - \alpha v_{t+1}$

- Let's set $\beta = 0.9$.

Momentum Example

- Iteration 1:

- - Without Momentum:

- $x_1 = 4 - 0.1 \cdot 2 \cdot 4 = 4 - 0.8 = 3.2$

- - With Momentum:

- $v_{t+1} = \beta \cdot v_t + (1 - \beta) \cdot \frac{\partial f}{\partial x}$

- $v_1 = 0 - 0.1 \cdot 2 \cdot 4 = -0.8$

- $x_{t+1} = x_t - \alpha v_{t+1}$

- $x_1 = 4 + (-0.8) = 3.2$

Iteration 2:

- Without Momentum:

$$x_2 = 3.2 - 0.1 \cdot 2 \cdot 3.2 = 3.2 - 0.64 = 2.56$$

- With Momentum:

$$\begin{aligned} v_2 &= 0.9 \cdot (-0.8) - 0.1 \cdot 2 \cdot 3.2 \\ &= -0.72 - 0.64 = -1.36 \end{aligned}$$

$$x_2 = 3.2 - 1.36 = 1.8$$

Momentum Example

- Iteration 3:
- - Without Momentum:
- $x_3 = 2.56 - 0.1 \cdot 2 \cdot 2.56 = 2.56 - 0.512 = 2.048$
- - With Momentum:
- $v_3 = 0.9 \cdot (-1.36) - 0.1 \cdot 2 \cdot 2.56 = -1.224 - 0.512 = -1.736$
- $x_3 = 1.84 - 1.736 = 0.104$
- As we can see from the example, gradient descent with momentum leads to faster convergence towards the minimum compared to standard gradient descent. The momentum term helps the optimization algorithm to maintain its direction and speed, resulting in smoother and more consistent updates, which can significantly improve convergence speed and stability, especially in complex optimization landscapes.

AdaGrad (Adaptive Gradient Algorithm)

- AdaGrad (Adaptive Gradient Algorithm) is an optimization algorithm designed to adaptively adjust the learning rate during training based on the frequency of feature occurrences.
- It was introduced by John Duchi, Elad Hazan, and Yoram Singer in 2011.
- The primary idea behind AdaGrad is to automatically scale the learning rates of individual model parameters based on the past gradients for each parameter.
- This adaptation helps in effectively handling sparse data and varying gradient magnitudes, leading to faster convergence and improved optimization performance.

AdaGrad (Adaptive Gradient Algorithm)

- The key is that AdaGrad effectively scales the learning rate based on the accumulated sum of squared gradients. Parameters with large gradients will have smaller effective learning rates, preventing aggressive updates, while parameters with small gradients will have larger effective learning rates, allowing faster convergence.
- This adaptive scaling helps AdaGrad navigate the optimization landscape more effectively, especially in scenarios with varying gradient magnitudes or sparse data.

AdaGrad algorithm:

- 1. Initialize Parameters: Initialize the model parameters θ and set the initial learning rate η .
- 2. Initialize Sum of Squared Gradients: Initialize a vector r to store the sum of squared gradients for each parameter. Each element r_i of this vector corresponds to the sum of squared gradients for parameter θ_i .
- 3. Iterative Optimization: - For each training iteration t : - Compute the gradient g_t of the loss function with respect to the model parameters θ . - Update the sum of squared gradients r by adding the square of the current gradient element-wise: $r = r + g_t^2$. - Compute the adaptive learning rates η_t for each parameter using the formula:

AdaGrad algorithm:

- $r = r + g_t^2$
- $\eta_{i,t} = \frac{\eta}{\sqrt{r_{i,t} + \epsilon}}$
- Here, ϵ is a small constant (e.g., 10^{-8}) added to prevent division by zero and improve numerical stability. - Update each parameter θ_i using the adaptive learning rate:
- $\theta_{i,t+1} = \theta_{i,t} - \eta_{i,t} \cdot g_{i,t}$
- 4. Repeat: Repeat the iterative optimization process until convergence criteria are met, such as reaching a maximum number of iterations or achieving a desired level of performance.

Advantages of AdaGrad:

- **Adaptive Learning Rates:** AdaGrad automatically adjusts the learning rates for each parameter based on their historical gradients. This adaptive scaling allows AdaGrad to effectively handle sparse data and varying gradient magnitudes.
- **Faster Convergence:** By adaptively scaling the learning rates, AdaGrad can converge faster than traditional optimization algorithms, especially in scenarios with uneven gradient distributions or noisy gradients.

Limitations of AdaGrad:

- **Diminishing Learning Rates:** AdaGrad's accumulation of squared gradients in the denominator can lead to diminishing learning rates over time. In some cases, this can result in early convergence or slow learning in the later stages of training.
- **Memory Requirements:** AdaGrad requires storing and updating additional variables (the sum of squared gradients) for each parameter, increasing memory requirements, especially for models with a large number of parameters.

RMSProp (Root Mean Square Propagation)

- RMSProp (Root Mean Square Propagation) is an adaptive learning rate optimization algorithm proposed by Geoffrey Hinton in his lecture slides in 2012.
- RMSProp addresses some limitations of AdaGrad by using a moving average of squared gradients to adjust the learning rates.
- The intuition behind RMSProp is to normalize the gradients based on their historical magnitude, preventing the learning rates from diminishing rapidly and providing better convergence properties, especially in non-convex optimization problems.

RMSProp (Root Mean Square Propagation)

- Instead of using all historical summation of squared gradient RMSProp suggest that we can use the exponential average for this summation
- The problem of adagrad when summation of squared gradient is very big can lead to the learning rate is very small and that lead to no update or very slow update to parameters so RMSProp solve this problem by consider the historical average of summing squared gradient

RMSProp (Root Mean Square Propagation)

- 1. Initialization: - Initialize the parameters θ and set the initial learning rate η . - Set a decay rate ρ (typically close to 1, e.g., 0.9) to control the smoothing of the moving average. - Set a small constant ϵ (e.g., 10^{-8}) for numerical stability.
- 2. Compute Moving Average of Squared Gradients: - At each iteration t , compute the squared gradients g_t^2 . - Update the exponentially decaying average of squared gradients v_t :
 - $v_t = \rho \cdot v_{t-1} + (1 - \rho) \cdot (g_t^2)$
- 3. Update Parameters: - Compute the adaptive learning rate:
 - $\eta_t = \frac{\eta}{\sqrt{v_t} + \epsilon}$
 - - Update the parameters using the adaptive learning rate:
 - $\theta_{t+1} = \theta_t - \eta_t \cdot g_t$

RMSProp (Root Mean Square Propagation)

- RMSProp divides the learning rate by a running average of the magnitudes of recent gradients. This helps to normalize the gradients and prevent the learning rates from diminishing too quickly.
- The moving average of squared gradients acts as a scaling factor that adapts the learning rates based on the historical magnitudes of the gradients.
- This allows RMSProp to handle noisy gradients
- The key intuition behind RMSProp is that it scales the learning rates based on the magnitudes of recent gradients, providing a smoother and more stable optimization process.
- By adapting the learning rates based on the historical gradients, RMSProp can converge faster and it be more reliable compared to using a fixed learning rate or AdaGrad, especially in scenarios with noisy or non-stationary(static) gradients.

RMSProp (Root Mean Square Propagation)

- One of the main problems with RMSProp is that it does not provide momentum, which can sometimes slow down convergence in certain scenarios, especially in high-dimensional optimization problems or those with uneven curvature.
- Momentum helps accelerate optimization by smoothing out updates and allowing the optimizer to continue in the same direction for parameters that consistently show improvement.
- Adam (Adaptive Moment Estimation) addresses this limitation by incorporating momentum into the optimization process while also maintaining adaptive learning rates like RMSProp.

Adam (Adaptive Moment Estimation)

- Adam (Adaptive Moment Estimation) is an optimization algorithm that combines the benefits of Momentum and RMSProp to efficiently navigate the optimization landscape.
- Introduced by D. P. Kingma and J. Ba in 2014, Adam is widely used in training deep neural networks and other machine learning models.
- In the Adam optimizer, the term “moment” refers to moving averages of the gradients, inspired by the concept of statistical moments (like mean (first moment), variance(second moment)).

Adam (Adaptive Moment Estimation)

- 1. Initialization: - Initialize model parameters θ . - Initialize first moment vector m (initialized to zeros). - Initialize second moment vector v (initialized to zeros). - Set hyperparameters: learning rate η , momentum decay parameter β_1 , RMSProp decay parameter β_2 , and small constant ϵ (for numerical stability).
- 2. Iterative Optimization: - For each iteration t : - Compute the gradient g_t of the loss function with respect to the parameters θ . - Update biased first moment estimate:
 - $m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$
 - - Update biased second raw moment estimate:
 - $v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot (g_t^2)$
 - - Correct bias in first moment:
 - $\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$

Adam (Adaptive Moment Estimation)

- - Correct bias in second moment:

- $\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$

- - Compute parameter update:

- $\Delta\theta_t = -\frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \cdot \hat{m}_t$

- - Update parameters:

- $\theta_{t+1} = \theta_t + \Delta\theta_t$

Adam (Adaptive Moment Estimation)

- 1. Adaptive Learning Rates: - Adam adapts the learning rates for each parameter based on the magnitudes of the gradients and their historical second moments. - The division by the square root of the second moment (RMSProp term) scales the learning rates adaptively for each parameter, allowing for faster convergence in regions with large gradients and slower updates in regions with small gradients.
- 2. Momentum: - Adam used momentum by maintaining a first moment estimate m that accumulates gradients over time. - The momentum term helps smooth out updates and allows the optimizer to continue moving in the same direction for parameters that consistently show improvement, leading to faster convergence and improved optimization performance.
- 3. Bias Correction: - Adam corrects biases in the first and second moment estimates to account for the initialization of m and v to zeros. - The bias-corrected estimates ($\hat{m}$ and $\hat{v}$) help ensure that the optimization process is unbiased and that the learning rates are appropriately adjusted.

References

- Linear Regression, ECE-5424G / CS-5824 Lecture Slides , Jia-Bin Huang , Virginia Tech <https://filebox.ece.vt.edu/~jbhuang/teaching/ECE5424-CS5824/sp19/index.html>
- ENG6600: Advanced Machine Learning ,“Optimization for ML: Gradient Descent: (Part 1)”, S. Areibi ,School of Engineering University of Guelph
https://islab.soe.uoguelph.ca/sareibi/TEACHING_dr/ENG6600_ML_html_dr/outline_eng6600/docs/LECTURE_dr/TOPIC1_dr/LECT_dr/ENG6500-1-IntroductionToMLDL-Part1.pdf
- [Hyper Parameter—Momentum. When we use the SGD \(stochastic... | by Yuanrui Dong | AI³ | Theory, Practice, Business | Medium](#)
- <https://stats.stackexchange.com/questions/232741/why-is-it-important-to-include-a-bias-correction-term-for-the-adam-optimizer-for>